דוח משוב לסטודנט

האוניברסיטה הפתוחה

מזהה סטודנט	N102567562
מספר מבחן	0001
תאריך בחינה	יום חמישי, 12 בפברואר 2026
מזהה קורס	20594
שם קורס	מערכות הפעלה
מרצה	אריה לויטן

תשובות נכונות (שאלות סגורות)	ניקוד שאלות סגורות	ניקוד שאלות פתוחות	ציון מבחן מקורי	ציון מבחן סופי
5	25.00	70.00	95.00	95.00

סיכום

מספר שאלה בטופס זה	מספר השאלה בטופס המקור	מיקום התשובה שסומנה בטופס המקור	ניקוד	ניקוד מירבי	התשובה שסומנה	התשובה הנכונה
1	1	1	5.00	5.00	1 ✓	1
2	2	3	5.00	5.00	3 ✓	3
3	3	1	5.00	5.00	1 ✓	1
4	4	2	5.00	5.00	2 ✓	2
5	5	4	0	5.00	4 ✕	2
6	6	2	5.00	5.00	2 ✓	2
7	7		25.00	25.00		
8	8		15.00	15.00		
9	9		15.00	15.00		
10	10		5.00	5.00		
11	11		5.00	5.00		
12	12		5.00	5.00		

✓ תשובה נכונה ✓ תשובה חלקית ✕ תשובה שגויה ⊘ לא נענתה ⓘ תשובה עודפת

האוניברסיטה
הפתוחה

דף תשובות

מספר גרסה	[redacted]	מספר הקורס	תאריך בחינה
0001	[redacted]	20594	120226

0	●	●	●	○
1	○	○	○	●
2	○	○	○	○
3	○	○	○	○
4	○	○	○	○
5	○	○	○	○
6	○	○	○	○
7	○	○	○	○
8	○	○	○	○
9	○	○	○	○

הוראות למילוי דף התשובות

1. נא למלא תשובה אחת בלבד על ידי השחרת העיגול - ●
2. את ההשחרה יש לבצע בעט שחור או כחול בלבד!
3. השימוש בטיפקס או מרקרים אסור!
4. כדי לתקן סימן, יש לסמן X על התשובה הלא נכונה - ⊗
ולהשחיר את התשובה הנכונה - ●
5. כדי לחזור לסימון המקורי, יש לסמן X על התשובה הלא רצויה,
ולהשחיר מחדש את סימן ה- ⊗ באופן שיכסה המחיקה - ⊗

שאלה / תשובה	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
א	●	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	○	○	●	⊗	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלה / תשובה	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלה / תשובה	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלון 510

5.20.85

ש מ N102567562

19 [redacted]

[redacted]

יש לרשום את כל תשע הספרות

האוניברסיטה
הפתוחה

כ"ה בשבט תשפ"ו

מס' שאלון - 510

12
בפברואר 2026

סמסטר 2026א

20594 / 4

מס' מועד 85

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **12** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלק א' (שאלות אמריקאיות) עליכם לבחור בכל פעם
בתשובה יחידה הנכונה ביותר מבין התשובות.
את התשובות בחלק א' יש לסמן בטופס המחשב המצורף לשאלון.

ג. בחלקים ב' ו ג' מופיעות שאלות פתוחות.
ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ.
אין חובה להשתמש בכל השורות המוקצות לצורך התשובות,
אך אין לחרוג מהמקום המוקצה.

חומר עזר:

כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.
אין להכניס חומר דיגיטלי!

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

הנחיות למבחן

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלק א׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות. **את התשובות בחלק א׳ יש לסמן בטופס המחשב המצורף לשאלון.**
2. בחלקים ב׳ ו ג׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות**, **בכתב קריא** ו**בקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (30 נקודות)

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולסמן את התשובה **בטופס המחשב המצורף לשאלון.**

שאלה 1

איזו פעולה מן הפעולות הבאות אפשר לבצע אך ורק במצב ראשוני (kernel mode) במערכת ההפעלה Linux?

[✓ א.] חסימת פסיקות החומרה (disabling hardware interrupts)

ב. החלפת תהליכונים (thread switch) כאשר מדובר בספריית תהליכונים ברמת המשתמש

ג. קבלת איתות(SIGNAL) מתהליך אחר

ד. הוספת פקודה חדשה לתוך תיקיית פקודות המערכת bin/

Containers run on GUEST

שאלה 2

בהשוואה בין וירטואליזציה וקונטיינריזציה, התייחסו למקרה בו על מערכת מארחת רצה מערכת אורחת וירטואלית עם אפליקציות שצריכות הרצה בתוך קונטיינרים. תבחרו בטענה הנכונה ביותר:

א. מנגנון תמיכה בקונטיינרים נדרש בשתי המערכות.

ב. מנגנון תמיכה בקונטיינרים נדרש רק במערכת מארחת כי מערכת אורחת, בהיותה וירטואלית, מסתמכת על תמיכה בקונטיינרים של מערכת מארחת.

[✓ ג.] מנגנון תמיכה בקונטיינרים נדרש רק במערכת אורחת כי יישום מנגנון קונטיינרים כלל אינו קשור למערכת מארחת.

ד. בשתי המערכות לא נדרשת תמיכה מיוחדת במנגנון קונטיינרים כי מנגנון קונטיינרים מהווה סוג של וירטואליזציה ולכן כלול מראש במערכת אורחת בהיותה וירטואלי ובמערכת מארחת בהיותה תומכת בוירטואליזציה.

שאלה 3

הניחו שקיימים 100 עותקים של משאב R במערכת וקיים מספר תהליכים שכל אחד מהם צריך עד ארבע עותקים של משאב R כדי להסתיים , חשבו כמה תהליכים כאלו **לכל היותר** יכולים להיות מופעלים במערכת כדי שיוכלו לסיים **בוודאות**, בלי שיתרחש קיפאון (deadlock)?

4

100 > X(3)

[✓ א.] 33

ב. 32

~~ג.~~ 96

~~ד.~~ 95

שאלה 4 ✗

נתונה מערכת עם זיכרון מדומה (virtual memory), המנוהל בשיטת "דפדוף לפי דרישה" (demand paging). נתון כי גודלו של כל דף הוא 512 בתים.

להלן הצהרה של מטריצה D: Double D[32][32];

8192 bytes

נתון כי גודלו של טיפוס הנתונים Double הוא 8 בתים. האיבר הראשון D[0][0] שמור בתחילת דף. נוסף על כך ,ידוע כי המטריצה שמורה באופן כזה שאיברי השורה הראשונה במטריצה שמורים לפני איברי השורה השנייה (row-order).

להלן קטע קוד:

```
for (int i = 0; i < 32; i++)
        for (int j = 0; j < 32; j++)
                D[ i ][ j ] = 0.0;
```

המטריצה לא נמצאת בזיכרון בתחילת הריצה. כמה שגיאות דף(page fault) יתרחשו אם למטריצה הוקצו ארבע מסגרות(frame) בלבד?

2 rows per page

א. 4

[✓ ב.] 16

ג. 32

ד. 64

שאלה 5 A

מהי הסיבה העיקרית לשימוש ב- DMA?

- א. למעבד דרוש פחות זמן לביצוע פקודה חישובית אחת.
- [✓ ב.] הקטנת הזמן שלוקח למעבד בסה"כ לטפל בפעולות קלט-פלט.
- ג. למעבד דרוש פחות זמן לביצוע גישה לזיכרון הראשי.
- ד. מאפשר להתקן, למשל דיסק, להשתמש בזיכרון הראשי כמטמון.

DMA does the work

שאלה 6

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node עם פרמטרים הבאים:

- גודל הבלוק במערכת הקבצים הוא 2 Kbytes 2^{11}
- כתובת הבלוק היא 4 בתים (bytes) 2^2
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

$2^9 = 512$

חשבו מה הגודל המרבי של קובץ שניתן לאכסן ב 1514 בלוקים בסה"כ(כולל נתונים ומצביעים, אבל לא כולל את ה I-NODE עצמו).

הערה: לצורך החישוב 1 KByte = 1024 Byte , 1 MByte = 1024 KByte

- א. 3028 Kbytes
- [✓ ב.] 3020 Kbytes
- ג. 3000 Kbytes
- ד. 3008 Kbytes

$\underline{10} + 1 + \underline{512} + 1 + 1 + \underline{512} + 1 + \underline{476}$

1510 blocks

המשך הבחינה בעמוד הבא

[redacted]

חלק ב (55 נקודות)

ענו על השאלות 7-9.

התשובות מרשימות מאוד! יש לך רקע בתיום? אם יש לך זמן,
תכתוב/` בבקשה למרכז ההוראה

שאלה 7 (25 נקודות)

חבל על טעות סימון(שונה בין שאלון לטופס) בשאלה סגורה 5.
ניתן לשקול הגשת ערעור מנומק

א. (16 נק')

סיימו את התוכנית להלן עבור בעיית קוראים-כותבים(כאשר קוראים יכולים לקרוא במקביל, כותבים לא יכולים לכתוב במקביל לא אחד עם השני ולא עם הקוראים). **הפתרון צריך לתת העדפה לכותבים לגבי מצטרפים חדשים("חדש" מוגדר כמי שעדיין לא הספיק לבצע את השורה הראשונה של הקוד).** הניחו **שכל תוכניות המשתמש מושהות עד סיום ביצוע קריאת מערכת.** הסמפורים הם אובייקטים של מערכת(כרגיל).

מלאו בכל שורה ריקה פעולה אחת בלבד(פקודה רגילה אחת או פעולה אחת על אמצעי סנכרון).

- semaphore BlockRead = 1; // binary semaphore used to block readers
- semaphore BlockWrite = 1; //binary semaphore used to allow ONE writer or MANY readers
- int Readers = 0, Writers = 0; // count of readers and writers in critical section
- semaphore RMutex = 1; // binary semaphore for Readers
- semaphore WMutex = 1; // binary semaphore for Writers

Reader

1. down(BlockRead); // at most one reader can go before a pending write
2. down(RMutex);
3. Readers++;
4. if (Readers == 1) // first reader acquire write lock
5. down(BlockWrite);
6. up(RMutex);
7. up(BlockRead);
8. < Do the Reading >
9. down(RMutex);
10. Readers--;
11. if (Readers == 0) // last (only) reader releases write lock
12. up(BlockWrite);

```
13. up(RMutex);
```

Writer

```
1.  down(WMutex);
2.  Writers++;
3.  if (Writers == 1)
4.      down(BlockRead);// block readers
5.  up(WMutex);
6.  down(BlockWrite);
7.  < Do the Writing >
8.  up(BlockWrite);
9.  down(WMutex);
10. Writers--;
11. if (Writers == 0)
12.     up(BlockRead);// enable readers
13. up(WMutex);
```

הסבירו כיצד הפרוטוקול עובד : האם בכלל ואיך מבטיח מניעה הדדית, התקדמות (העדר קיפאון) והמתנה מוגבלת.

מובטח מניעה הדדית בגלל Blockwrite/Read: אף קורא לא יכול לקרוא בזמן שכותב כותב וכותבים לא יכולים לכתוב במקביל. כשקוראים מעוניינים לקרוא, הראשון מביניהם אחראי לתפוס את Blockwrite מה שימנע מכותבים לכתוב (כותבים-שורה 6) וכשכותבים מעוניינים לכתוב, הראשון מביניהם אחראי לתפוס את BlockRead מה שימנע מקוראים לתפוס אפילו את השורה הראשונה שלהם (כך נכנס ענין העדיפות). הסמפורים הבינאריים W/RMutex מגנים על המונים של מספר הכותבים/קוראים. מובטח התקדמות מכיוון שאחד מהתנאים ההכרחיים לקיפאון הוא המתנה מעגלית מה שלא מתרחש כאן: כשתהליך חסום על סמפור, התהליך שמחזיק את הסמפור הזה לא בעצמו חסום על סמפור שהתהליך הראשון מחזיק. כלומר כשקוראים חסומים על BlockRead כי הוא נתפס ע"י הכותבים, הכותבים או כבר תפסו את Blockwrite או מחכים שהקוראים שכבר נכנסו (כבר קוראים) ישחררו אותו (מה שמובטח שיקרה בסופו של דבר). כנ"ל הפוך - כשכותבים חסומים על BlockRead כן לא מעגל המתנה משולב סיבה. מובטח המתנה מוגבלת מכיוון שבין כותבים לאחר שכותב אחד מסיים את הכתיבה שלו הוא משחרר את Blockwrite לכותבים האחרים ולאחר שהכותבים/קוראים סיימו כ"קבוצה" הם משחררים את BlockWrite/BlockRead לקבוצה השנייה.

ב. (9 נק')

הסבירו באיזו מידה ואיך מנגנון הסנכרון מבטיח העדפת הכותבים.

25
(7)

תשובה והסבר:

הכותבים מקבלים עדיפות בגלל הסמפור BlockRead, בניגוד לקוראים שמשחררים את הסמפור בין קוראים (כלומר כל קורא אחראי לתפוס ולשחרר את BlockRead בעצמו), הכותבים לא משחררים את BlockRead כל עוד יש כותב שמחכה לכתוב. דבר זה מבטיח שרק הקוראים שכרגע מבצעים קריאה (ביצעו כבר את שורה 7) יוכלו להמשיך בפעולתם ועוד שקוראים חדשים יצטרכו לחכות עד שהכותב האחרון יסיים את כתיבתו. הקוד עצמו זה ~~[illegible]~~ ההבדל בין שורה 7 בקוראים לשורה 12 בכותבים ושני כל אחד מהם משמש (כחלק מ-"זוג" או לא). דבר זה גורם לכך שכותבים "חדשים" מקבלים עדיפות על קוראים "חדשים" גם אם הקוראים הגיעו (ניסו לבצע את שורה 1 שלהם) קודם. לכן יש עדיפות כנדרש.

שאלה 8 (15 נקודות)

א. (5 נק')

בהתייחס למעבד מרובה ליבות(multi-core), הסבירו את היתרון והחיסרון העיקרי בתזמון זיקה (Affinity scheduling) מבחינת מהירות ביצוע הקוד(כולל מהירות גישה למשתנים).

תשובה והסבר :

תזמון זיקה מתבסס על הרעיון שעדיף להריץ תהליכים על אותה ליבה שהם רצו בה בעבר. היתרון של תזמון זה הוא שיפור בביצועי ההרצה (כלומר לוקח פחות זמן) מכיוון שאם מריצים על אותה ליבה אזי ה- cache וה-TLB "רלוונטיים" ולא כל פעם מתחילים מאפס. לכן מהירות גישה למשתנים משתפרת מאשר אם מריצים על ליבה אחרת (נשמר כבר ב-cache). החיסרון המרכזי הוא שתזמון זה עלול ליצור עומס: אם יש ליבה אחת שלא מספיקה להריץ את כל התהליכים שלה וליבה אחת שפנויה, זה ~~[illegible]~~ הבזבוז זמן והעומס לא מחולק בין הליבות.

ב. (5 נק')

בהנחה שכל ליבה core כוללת את הרכיבים העיקריים של מעבד שלם, הסבירו האם Affinity scheduling משפיע על פספוסי TLB(TLB miss)?

Translation Lookaside Buffer

תשובה והסבר :

כן. כפי שציינו בסעיף הקודם, אם מריצים תהליכים על אותם ליבות שהם רצו בעבר אזי ה- TLB שלהם עדיין "רלוונטי" כלומר לא כל פעם כל ליבה צריכה למלא את ה- TLB מחדש עבור כל תהליך אלא של הליבה של התהליך הזה כבר שמר cache של ה-"VA→PA translations" ולכן יהיו פחות TLB miss ולכן פחות ~~[illegible]~~ גישות לטבלת הדפים (ולכן ביצועים יותר מהירים).

ג. (5 נק')

15
(8)

בהמשך לסעיף א, הסבירו האם Affinity scheduling מפחית את תקלות עמוד(page faults)?

תשובה והסבר :

לא, תזמון זיקה לא משפיע על כמות ה-page faults מכיוון שה-page faults מתבצעים כאשר ניגשים לדף חדש ו/או נגמר המקום בזיכרון הראשי אזי התבצע swap והדף שלנו "ירד" ל- swap file ועכשיו אנחנו צריכים אותו שוב. בכל מקרה הסיבות שבהם קורה page fault לא תלויות באיזה ליבה מריצים אותנו מכיוון שבניגוד למטמון או ל- TLB, כל הליבות חולקות את הזיכרון הראשי בצורה כזו או אחרת (או UMA או NUMA) ולכן כמות ה- page faults לא ~~[illegible]~~ מושפעת מאיזה ליבה מריצה את התהליך.

שאלה 9 (15 נקודות)

א. (5 נק')

האם הקצאת מקום לקבצים בעזרת אינדקס(I-Node) מתאימה מבחינת יעילות השימוש לדיסק לא מכני SSD? Solid State drive

התייחסו בנפרד לקריאה וכתיבה של הנתונים
כן   לא (יש פתרון)

תשובה והסבר:

15
(9)

ל-SSD יש קריאה וכתיבה "אסימטריים": הקריאה מאוד מהירה ואין הבדל משמעותי בין קריאה רציפה לקריאות אקראיות וגם שהכתיבה משמעותית יותר איטית, במיוחד הכתיבות של מעט מידע בצורה אקראית. לכן I-Nodes מתאימים לקריאה ב-SSD אבל כתיבה פחות מתאימה. ה-INodes לא יכולים להכנס למצב של כתיבות קטנות אקראיות. "הרקורסיביות" כשמעדכנים בלוק של INode ולכן אולי צריך לעדכן indirect point וכו... לכן מה שעושים זה מיפוי "גלובלי" בין מספר לוגי של בלוק לכתובת יזית בדיסק וה-FTL אחראי לבצע translation מכתובת ליניארית של INodes לכתובת היררכית של SSD (הכתובת חייבת להיות היררכית ב-SSD מכיוון ש-flash packages מורכבים מ-dies שמורכבים מ-planes שמורכבים מיחידות מחיקה שמורכבה מיחידות קלט/פלט).

Flash Transla Layer

ב. (10 נק')

הסבירו איך מערכת קבצים מבוססת יומן log יכולה ליעל את שימוש בדיסק SSD.

תשובה והסבר:

ב-SSD לא ניתן לדרוס יחידת קלט/פלט אלא ע"מ לכתוב שוב למקום שכבר נכתב עליו בעבר נדרש להעתיק את כל יחידת המחיקה שמכילה את יחידת הקלט/פלט הרצויה למקום אחר פנוי בדיסק, למחוק את יחידת המחיקה, ואז להעביר בחזרה את המידע "חוקי" בסדר כלומר נדרש המון עבודה (לכן כתיבות קטנות במקומות אקראיים זה יקר ב-SSD). log-structured FS מיעלה ל-SSD מכיוון שבמקום להעתיק ולמחוק ולהעתיק שוב, פשוט מגדירים את יחידת הקלט/פלט כלא שמיש יותר וכותבים בהמשך ה-"log" את העדכון כלומר זה ממש נראה כמו קובץ עדכונים. אח"כ ה-"garbage collection" של ה-FTL יטפל ביחידת הק/פ שהוגדר כלא בשימוש (או כתגובה לפקודת TRIM של מ"ה ל-SSD).

בנוסף, log-structured FS מפזר את הכתיבות ב-SSD וזה ממש חשוב מכיוון שלכל "תא" ב-SSD יש הגבלה בכמות כתיבות שניתן לבצע בו ולכן חשוב לא לשחוק אזור אחד ב-SSD לפני אחרים כלומר נדרש לפזר את הכתיבות, מה שאכן קורה ב-log, (בנוסף לכך שהוא נמנע מכתיבות דריסה יקרות). לסיכום ה-FS הזה מיטב ל-SSD ב-2 סיבות חשובות:

① נמנע מכתיבות קטנות אקראיות 9 שיקרות (הסברתי למה הן יקרות)

② מפזר את הכתיבות ב-SSD בגלל הגבלת אורך חיים של תא

חלק ג (15 נקודות)

שאלות 10-12.

(5 נקודות לשאלה ; 15 נקודות לחלק זה).

שאלה 10

5
(10)

התייחסו ל 3 מושגים : קריאת מערכת עצמה(TRAP), מעטפת קריאת מערכת (לדוגמא Write) ופונקצית ספריה(לדוגמא Printf) והסבירו בעקבות ביצוע מה נעשה מעבר ממצב משתמש (USER MODE) למצב ראשוני(KERNEL MODE)? ← TRAP

תשובה והסבר :

פונקצית ספריה היא מעטפת נוחה למשתמשים שמקבלת למשל פרמטרים ומבצע formatting והדפסת למסך למשל. כך בעצם אוסף של פונקציות שימושיות. שימושיות למשתמשים בצורה נוחה והם OS-independent (אותה ספריה גם ב-windows וגם ב-Linux). מעטפת קריאות מערכת היא כזאת ששמה ברגיסטר %eax את מספר ה-syscall (מה שאפשר לגרעין לדעת איזו פעולה לבצע לאחר הקריאה ל-TRAP). TRAP (למשל int 0x80) הוא זה שמעביר את השליטה ממצב משתמש למצב גרעין, קורא מ-%eax את מספר ה-syscall ולפי זה גם יודע כמה args הוא אמור לקרוא ומה כל אחד מהם מ"צג.

שאלה 11

מה יהיה הפלט אחרי הרצת הקוד הבא במערכת XV6 שתומכת בקונטיינרים?

ידוע שה PID הגדול ביותר שהיה במערכת לפני הרצת הקוד היה 15.

ניתן להניח שכל קריאות מערכת מצליחות.

```
16 pid = fork();
   if (pid > 0)  {   אבא
      unshare(PID_NS);
      pid = fork(); 17/1
      if (pid > 0) printf (stdout, "Parent's perspective on his child: PID=%d \n",
17 pid);
      else  {
                                                                          0
         printf (stdout, " Child's perspective on his parent: PID=%d \n", getppid());
         printf (stdout, " Child's perspective on himself: PID=%d \n", getpid());
                                                                        1
      }
בן else printf(stdout, "Son perspective on himself: PID=%d \n", getpid);
                                                                16
```

תשובה והסבר:

5
(11)

"Parent's... PID=17", "Child's... parent: PID=0",
"Child's... himself: PID=1", "Son... PID=16"

האבא יוצר 2 ילדים: "son" עם PID 16 ו-"child" שבתוך PID namespace ולכן יש לו PID גלובאלי של 17 ולוקאלי של 1. תהליך בתוך PID_NS מקבל את ה-PID הלוקאלי שלו כשקוראים ל-getpid() ולכן מודפס 1 (הוא התהליך הראשון ב-PID_NS החדש). תהליך INIT שקורא ל-getppid() מקבל בחזרה 0 כי הוא ה-INIT של הקונטיינר ולכן מודפס 0. האבא מחוץ לקונטיינר ולכן מדפיס את ה-PID הגלובאלי של Child, (17).

שאלה 12

א. מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים

ב. כמה קבצים file.txt ייוצרו בסה"כ בדיסק?

```
int pid = fork();
if (pid == 0) {
  unshare (MOUNT_NS);
  mkdir ("MDirA");
  mount ("internal_fs_a","MDirA",0); //mount a preformatted internal_fs_a
  mkdir ("MDirB");
  mount ("internal_fs_b","MDirB",0); //mount a preformatted internal_fs_b

  int fd1 = open ("MDirA/file.txt",O_CREATE|O_RDWR);
  write (fd1 ,"Some text\n",10);
  close (fd1);
  int fd2 = open ("MDirB/file.txt",O_CREATE|O_RDWR);
  write (fd2 ,"Some text\n",10);
  close (fd2);

 }
If (pid>0) {
   sleep (10000); // wait for child to finish
   if (open ("MDirB/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to
open\n");
   else printf (stderr, "father succeeded\n");
}
if( pid == 0) {
   if (open ("MDirA/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to
open\n");
   else printf (stderr, "son succeeded\n");
 }
```

תשובה והסבר:

namespace mount זה הפרדה בנקודות חיבור של מערכת קבצים. כשהבן מבצע unshare() הוא יוצר עותק פרטי של ה-mount table של ה-FS הנוכחי אבל ה-mount יוצר את "הקישור" עצמו, לכן יודפס "father failed"⊛ ו- "son succeeded". (את השינוי ב-mount table עצמו כלומר כבר לא אותה טבלה כמו של האבא). ייוצרו 2 file.txt בדיסק, אחד בכל mount (ניתן לעשות mount ל-dir-ים שונים).

5 (12)

בהצלחה!

⊛ אבל אם הבן לא היה נעשה mount לאחר ה-unshare, האבא + בן היה נגיש לקובץ מכיוון שה-unshare לא משנה כלום, רק יוצר עותק כפי שכתבתי בהתחלה.